

DataScience Copilot — Poora Project, Ek Jagah Pe

Code padh ke likha gaya breakdown — folder names se nahi. Har claim real code se aati hai. Jahan kuch unclear ya unfinished hai, wahan saaf saaf likha hai.

LangGraph

FastAPI

Multi-Agent

Self-Healing Pipeline

Agar tumhe iss project ke baare mein kuch nahi pata — koi baat nahi. Section 1 tumhe zero se samjhayega ki ye kya hai, kaise kaam karta hai, aur kyun banaya gaya. Section 2 mein 38 real interview questions hain, hints ke saath.

Ye Project Ko Samjho

Kya karta hai (seedhi bhasha mein)

- Tum apna dataset upload karte ho (CSV/Parquet/Excel) aur simple bhasha mein apni problem batate ho, jaise “predict churn”.
- Backend khud hi poora ML pipeline run kar deta hai: data ko profile karta hai, problem type aur target column figure out karta hai, data clean karta hai, features banata hai, kai models train karke best wala pick karta hai, held-out data pe honestly score karta hai, explain karta hai, aur ek report generate kar deta hai.
- Agar koi step beech mein fail ho jaaye, toh system khud hi problem diagnose karke fix karta hai aur usi step ko retry karta hai — bas crash nahi hota.

SABSE INTERESTING PART

Isme sabse interesting cheez **self-fix loop** hai. Poore project mein sabse zyada isi ko deeply samajhna zaroori hai — interview mein bhi yehi sabse zyada poocha jaayega.

Problem kya solve hota hai, aur kiske liye hai

- **Kiske liye:** jiske paas tabular dataset hai aur ek sawaal hai, aur jo har baar poora scikit-learn pipeline hath se nahi likhna chahta.
- **Dikkat:** har baar wahi boilerplate likhna — profiling, cleaning, encoding, cross-validation, model choose karna, honestly evaluate karna, results explain karna.
- **Solution:** LLM decide karta hai ki kya karna hai (plan, target, feature choices), aur plain deterministic Python *poora* actual ML kaam karta hai. LLM kabhi khud code nahi likhta ya run nahi karta.

CORE DESIGN IDEA — YAAD RAKHNA

LLM decide karta hai, tools execute karte hain. Almost har design question isi pe wapas aata hai, isliye ye line dimaag mein fix kar lo.

Kaise kaam karta hai — step by step (real names ke saath)

Pipeline ek LangGraph graph hai. Nodes fixed hain; unke chalne ka *order* har run ke plan se aata hai.

- 1 Upload** — `POST /api/datasets/upload` (`api/routes/datasets.py`). DET
 Extension aur size check hota hai (`max_upload_mb = 200`), phir accept karne se pehle `profile_dataset` se file actually parse ki jaati hai. Jo file parse nahi hoti, wo reject ho ke delete ho jaati hai, store nahi hoti.
- 2 Run create karna** — `POST /api/runs` → `create_run` (`api/routes/runs.py`). DET
 State seed karke `graph.invoke(...)` call karta hai. **Ye call synchronous hai** — matlab jab tak poora pipeline finish nahi hota, HTTP request block rehta hai (Limitations mein dekho).
- 3 profile_node** — Deterministic. DET
`profile_dataset` run karta hai, `profile.json` cache karta hai. Koi LLM nahi.
- 4 planner_node** — LLM. LLM
`PlannerAgent` capability names ka ek ordered list propose karta hai. `generate_plan` uske baad `validate_plan` run karta hai. Agar invalid ho, toh validator ka error text LLM ko wapas diya jaata hai aur wo retry karta hai, max `plan_attempts = 3` baar. 3 tries ke baad bhi valid plan na mile toh status `needs_input` ho jaata hai (bad plan run karne ke bajaye human ke liye ruk jaata hai).
- 5** DET

`route` — Ek pure function (`planner/planner_node.py`).

Har node ke baad state dekh ke decide karta hai ki agla node kaunsa chalega: plan ko cursor se follow karna, ya `reflect` / `planner` pe divert karna, ya stop karna. Yehi executor hai.

6 `insights_node` (optional) — LLM (`EDAInsightsAgent`).

LLM

Iska kuch depend nahi karta; skip ho jaaye toh bhi run fine chalta hai.

7 `problem_spec_node` — LLM (`ProblemSpecAgent`).

LLM

`problem_type`, `target_column`, `drop_columns` decide karta hai. Agar LLM ek aisa target column bataye jo exist hi nahi karta, toh `ValueError` raise hota hai.

8 `cleaning_node` — Deterministic (`clean_dataset`).

DET

Columns drop karta hai, duplicates drop karta hai, `cleaned.parquet` likhta hai.

9 `feature_plan_node` — LLM (`FeatureEngineeringAgent`).

LLM

Important baat: ye **cleaned** data ko profile karta hai, raw data ko nahi, isliye skew aur dtypes wahi reflect karte hain jo actually bacha hai.

10 `features_node` — Deterministic (`engineer_features`).

DET

Sirf row-wise transforms yahan hote hain (datetime expansion, log transforms, drops). `features.parquet` likhta hai.

11 `training_node` — Deterministic (`train_models`).

DET

Candidate models ko cross-validate karta hai. Spec se `cv_folds` read karta hai, default `CV_FOLDS = 5`. Winner ko poore data pe refit karta hai, `model.joblib` save karta hai.

12 `evaluation_node` — Deterministic (`evaluate_model`).

DET

Fir se split karta hai (`TEST_SIZE = 0.2`), train split pe refit karta hai, held-out split pe honest metrics ke liye score karta hai.

13 `explain_node` — Deterministic (`explain_model`).

DET

Agar available ho toh SHAP, nahi toh permutation importance, nahi toh khud ko "unavailable" mark kar leta hai.
Kabhi raise nahi karta.

14 `recommendations_node` — LLM (`RecommendationsAgent`).

LLM

Metrics + drivers ko business suggestions mein convert karta hai (max `MAX_RECOMMENDATIONS = 6`).

15 `report_node` — Deterministic (`assemble_report`).

DET

`report.md` likhta hai.

16 `summarize_node` — Plain Python, LLM nahi.

DET

Text summary banata hai, `status = "completed"` set karta hai. Agar insights skip huye ho toh gracefully handle karta hai.

17 **Result** — `create_run` ka final step.

DET

`run.json` mein poora audit trail likhta hai (plan, reports, reflection history). Frontend isko read karke report render karta hai.

Self-fix loop (ye wala part sabse zyada important hai)

Har capability node `_make_capability_node` (`graph/builder.py`) se wrap hota hai. Ye wrapper **saare exceptions catch karta hai** aur state mein `plan_error`, `failed_capability`, `failed_exc_type` likh deta hai — graph ko kabhi crash nahi hone deta. Uske baad:

1. `route plan_error` dekh ke run ko `reflect_node` (`reflection/node.py`) pe bhej deta hai.
2. `reflect_node` pehle budgets check karta hai: `reflection_attempts` (2 per capability) aur `repair_attempts` (4 per run). Dono mein se koi khatam ho gaya toh planner ko escalate ho jaata hai.
3. Warna ye diagnose karta hai: `ReflectionAgent.diagnose` pehle `taxonomy.classify` (pure regex rules) run karta hai. Agar confidence `HEURISTIC_MIN_CONFIDENCE = 0.6` se zyada ho, toh heuristic pe trust karta hai. Nahi toh LLM diagnosis pe fallback karta hai — jo `ThreadPoolExecutor` ke andar `diagnosis_timeout_s = 20.0` timeout ke saath chalta hai. **Diagnosis kabhi raise nahi karta.**
4. `select_repair` diagnosis ko ya toh concrete repair ya escalation pe map karta hai.
5. Agar repair hai, toh `REPAIR_REGISTRY` (9 registered) se ek runner artifact ko fix karta hai — usually `tools/repair_ops.py` mein ek pure primitive se `features.parquet` ko **in place** rewrite karta hai.
6. Agar repair se actually kuch change hua, toh `reflect_node` error clear karke `pending_retry_capability` set kar deta hai. `route` fir usi capability ko re-run karta hai. Reflection ke dauran plan cursor kabhi move nahi hota.
7. Agar repair no-op tha, ya concern “planning” wala hai (model config, resource limit), toh planner ko escalate ho jaata hai.

MOTIVATING EXAMPLE (GOLDEN TEST BHI HAI)

Training fail hoti hai `could not convert string to float: 'Yes'` error ke saath. Taxonomy ise `TARGET_ENCODING_ERROR` classify karta hai (confidence 0.9) → `label_encode_target` repair chalta hai → `Yes/No` target column `0/1` ban jaata hai → training retry hoti hai → success ho jaata hai.

Text architecture diagram



Upload → FastAPI → LangGraph (planner + route) → LLM agents / deterministic tools → self-fix loop → checkpointed state

WHITEBOARD PE STRESS KARNE LAYAK

- **Graph static hai; plan dynamic hai.** Nodes ek baar register hote hain; `route` + plan mil ke har run ka path decide karte hain.
- **State references rakhta hai, payloads nahi.** DataFrames aur models disk pe rehte hain; `RunState` sirf file paths aur chhote report dicts carry karta hai. Isi wajah se har node ke baad SQLite checkpointing cheap padti hai.

Tech stack — aur har piece kyun hai (project-specific)

BACKEND

Piece	Kyun hai
FastAPI	API surface hai (<code>/datasets</code> , <code>/runs</code>). Pydantic se typed request/response models bhi free mein mil jaate hain.
LangGraph	Pipeline ek graph hai jisme conditional-edge router aur ek checkpointer hai. Specifically do features ke liye: <code>route</code> -driven dynamic ordering, aur har node ke baad <code>SqliteSaver</code> checkpointing taaki run resume ho sake.
Pydantic	Har LLM agent ek Pydantic model return karta hai, aur LLM ko us schema ko match karne ke liye <i>force</i> kiya jaata hai. <code>Settings</code> env se <code>APP_</code> prefix ke saath saare knobs read karta hai.
pandas / numpy	Saara data ka kaam: profiling, cleaning, feature ops, repair primitives.
scikit-learn	Leak-safe preprocessing ke liye <code>Pipeline</code> + <code>ColumnTransformer</code> , <code>cross_val_score</code> , <code>train_test_split</code> , metrics, <code>permutation_importance</code> .
XGBoost + LightGBM	Classification aur regression dono mein 4 candidate models mein se do.
SHAP	Model explanations. Guard (<code>_HAS_SHAP</code>) ke peeche import hota hai, taaki missing/broken SHAP install pe run break na ho — permutation importance pe fallback.
joblib	Fitted pipeline save/load karta hai (<code>model.joblib</code>).
pyarrow / parquet	Intermediate artifacts (<code>cleaned.parquet</code> , <code>features.parquet</code>) Parquet format mein hain: typed, compact, retry pe fast re-read.
OpenAI SDK	Sirf OpenRouter ke liye HTTP transport ke tor par (<code>base_url="https://openrouter.ai/api/v1"</code>). Project OpenAI models se bandha nahi hai.

FRONTEND

Piece	Kyun hai
Next.js 14 / React 18	Upload + report UI. <code>hero-section.tsx</code> upload aur run karta hai; <code>report-view.tsx</code> leaderboard, held-out metrics, SHAP bars, recommendations, download buttons render karta hai.
framer-motion / three.js / tailwind / lucide-react	Presentation ke liye (animation, 3D hero, styling, icons). ML story mein central nahi hain.

Genuinely interesting engineering decisions

1 LLM plan karta hai, par sirf ek fixed menu se — aur ek validator har cheez run hone se pehle check karta hai.

Planner sirf wahi names emit kar sakta hai jo `REGISTRY` (11 capabilities) mein exist karte hain. `validate_plan` phir plan ko ek DAG ki tarah check karta hai: names registered hain, har step ke `requires` pehle ke `produces` se satisfy ho rahe hain, koi artifact do baar produce nahi ho raha, aur terminal artifacts (`summary` , `report`) reach ho rahe hain. Bad plan reject ho jaata hai aur error LLM ko wapas feed hota hai. Isliye LLM ki freedom dono taraf se boxed hai — na wo koi step invent kar sakta hai, na hi unsound order emit kar sakta hai.

2 Kisi provider ke JSON mode pe depend kiye bina universal structured output.

`OpenRouterClient.complete(system, user, schema)` prompt mein `schema.model_json_schema()` embed karta hai, ek single JSON object maangta hai, code fences strip karta hai, aur `schema.model_validate_json` se validate karta hai. `ValidationError` aane pe error wapas feed karke max `MAX_VALIDATION_RETRIES = 2` baar retry karta hai. Ye kisi bhi model ke saath kaam karta hai jo OpenRouter ke peeche hai, sirf native structured output wale models tak limited nahi.

3 Self-fix layer "data fix karo" aur "plan change karo" ko clearly separate rakhta hai.

`select_repair` in dono mein se exactly ek return karta hai: ek `RepairSpec` (artifact rewrite karo, same step retry karo) ya escalation (planner ko hand over karo). Data/encoding/NaN/dtype problems repair ho ke retry hoti hain. Model-config aur resource problems escalate hoti hain. Ye boundary ek hi jagah enforce hoti hai, isi wajah se loop samajhna easy hai.

4 Diagnosis heuristic-first, LLM-second, aur time-boxed hai.

`taxonomy.classify` exception type + message pe pure regex hai — fast, free, deterministic. LLM sirf tab consult hota hai jab heuristic unsure ho (confidence < 0.6), aur tab bhi 20-second timeout ke andar chalta hai, heuristic fallback ke saath. Isliye common cases ka cost zero hai aur graph kabhi slow model pe hang nahi hota.

5 Design se hi leak-safe.

`features_node` tool sirf row-wise transforms karta hai. Jo bhi cheez *ek parameter seekhti hai* — imputation values, scaling stats, one-hot categories — wo sab sklearn `Pipeline` ke andar rehti hai, isliye cross-validation ke dauran per-fold fit hoti hai. Koi alag "poore data pe scaler fit karo phir split karo" wala step nahi hai jo information leak kare.

Honest limitations (jo code mein actually hai)

Interviewer ko impress karna hai toh ye section sabse zyada kaam aayega — khud apne limitations pata hona seniority dikhata hai.

1. `graph.invoke()` **synchronous aur blocking hai (`runs.py:106`).**
`POST /api/runs` ka HTTP request poore pipeline ke liye open rehta hai — profiling, training, sab kuch. Koi job queue ya background worker nahi hai. Real load mein sabse pehle yahi tootega, aur "ye scale kaise karega" ka honest answer yehi hai.
2. **Planner escalation abhi actually strategy change nahi karta.**
Jab reflection kuch fix nahi kar paata, toh planner ko escalate hota hai. Par `_handle_escalation` abhi bas **same plan ko same failed step se retry karta hai**, aur `replan_attempts = 2` ke baad `needs_input` pe give up kar deta hai. `planner_hint` (jaise "model swap karo", "search space reduce karo") warnings mein record hota hai par **act nahi hota**. Escalation branch ko intelligent replan mat samajhna; ye abhi ek graceful give-up hai.
3. **`validate_schema` repair mein ek chicken-and-egg limit hai.**
Ye feature columns ko `training_report.feature_columns` ke saath realign karta hai — jo tabhi exist karta hai jab training ek baar succeed ho chuki ho. First-ever training failure pe align karne ke liye kuch hota hi nahi, isliye ye no-op ho jaata hai aur layer escalate ho jaati hai. Crash nahi karta, gracefully degrade karta hai, par jis case ki tum sabse zyada expect karoge usme help nahi kar paata.
4. **Single-process, single-machine.**
State checkpoints ek local SQLite file (`checkpoints.db`) mein jaate hain aur artifacts local filesystem (`storage/`) mein. Machines ke beech kuch bhi shared nahi hai. Ek box ke liye fine hai; horizontal scaling ke liye as-is built nahi hai.
5. **End-to-end self-heal sirf ek path ke liye tested hai, sab ke liye nahi.**
`test_router_resume.py` poore compiled graph ko ek failure → reflect → repair → retry → `completed` se drive karta hai — par ye path `retry_llm` (transient) repair use karta hai. Artifact-rewriting repairs (`label_encode_target` , etc.) node-level aur pure-function unit tests se proven hain, poore graph se drive nahi kiye gaye.
6. **Ek knob value ek human-readable string ke andar chhupi hai.**
`reduce_cv_folds detail="reduce cv_folds 5 -> 4"` return karta hai, aur `reflect_node._parse_trailing_int` naya fold count us string ke `end` se nikal leta hai. Kaam karta hai aur tested hai, par us detail string ki wording effectively ek contract hai — thoda fragile coupling.
7. **Frontend: sirf main path trace kiya.**
`page.tsx` , `hero-section.tsx` , aur `report-view.tsx` real working flow hain. Repo mein aur bhi section components hain; fully verify nahi kiya ki wo kaise (ya kya) wired hain.
8. **README code se thoda peeche hai.**
Isme frontend ko "pending" bataya gaya hai (jabki wo ab exist karta hai) aur `repairs.py` ke comments "Phase E" ko future work bolte hain, jabki `cv_folds` knob actually fully wired hai. Interview se pehle jaan lena — minor doc drift hai.

38 Interview Questions — 7 Categories

Hints sirf direction dete hain; poora answer nahi hain. Zor se bol ke practice karo, har question ka.

1. Pura flow samjhao

1.1 “Upload se le ke final report tak, pura flow samjhao.”

Hint: upload parse karke validate karta hai → `create_run` → profile → planner+validator → route plan ko walk karta hai → deterministic tools + LLM agents → summarize `completed` set karta hai.

1.2 “LLM actually kaha use hota hai, aur kaha nahi hota?”

Hint: LLM planner + 4 agents mein hai (problem/feature/insights/recommendations); baaki sab deterministic tools hain; LLM kabhi code run nahi karta.

1.3 “Steps ka order kaun decide karta hai?”

Hint: `PlannerAgent` names propose karta hai, `validate_plan` DAG check karta hai, `route` plan ko cursor se walk karta hai.

1.4 “System ka sabse interesting flow dikhao.”

Hint: `Yes/No` target failure → taxonomy `TARGET_ENCODING_ERROR` → `label_encode_target` → retry → success.

1.5 “User ko end mein kya milta hai?”

Hint: `run.json` (plan, reports, reflection history) + downloadable `model.joblib` aur `report.md`.

2. Architecture aur design

2.1 “LangGraph hi kyun, plain function pipeline ya Celery chain kyun nahi?”

Hint: Dynamic order aur retry loops ke liye conditional-edge routing, aur resume ke liye per-node checkpointing.

2.2 “Graph static hai par plan dynamic — kaise?”

Hint: Nodes `build_graph` mein ek baar register hote hain (`@lru_cache`); `route` + plan har run ka path pick karte hain.

2.3 “State DataFrames ki jagah file paths kyun rakhta hai?”

Hint: `RunState` aur har checkpoint ko chhota rakhne ke liye; heavy data Parquet/joblib ke tor par disk pe rehta hai.

2.4 “LLM ko impossible plan banane se kaise roka jaata hai?”

Hint: Registry-bounded names + `validate_plan` DAG check (requires/produces/duplicates/terminal) + feedback-retry.

2.5 “Artifact repair karo aur planner ko escalate karo ko alag kyun rakha?”

Hint: Data problems in-place retryable hote hain; strategy problems ko alag plan chahiye; `select_repair` ek-ya-doesra enforce karta hai.

2.6 “Provider ke JSON mode ke bina structured output kaise kaam karta hai?”

Hint: Schema-in-prompt + `model_validate_json` + `OpenRouterClient` mein bounded validation-retry.

2.7 “OpenAI ka nahi use karte, phir OpenAI SDK kyun hai?”

Hint: Ye sirf OpenRouter ke OpenAI-compatible endpoint ka transport hai; abstraction `LLMClient` hai.

3. Deep-dive code

3.1 “Jab koi capability node raise karta hai, exactly kya hota hai?”

Hint: `_make_capability_node` isko catch karta hai, `plan_error` / `failed_capability` / `failed_exc_type` likhta hai, kabhi re-raise nahi karta.

3.2 “`reflect_node` ka decision order samjhao.”

Hint: Budgets check karo → diagnose karo → `select_repair` → repair run karo → agar change hua toh retry, nahi toh escalate; cursor kabhi advance nahi hota.

3.3 “Diagnosis slow LLM pe hang hone se kaise bachta hai?”

Hint: Pehle heuristic (≥ 0.6 pe trust); LLM fallback `ThreadPoolExecutor` ke andar `diagnosis_timeout_s = 20.0` ke saath; timeout pe heuristic return karta hai.

3.4 “Training ke dauran data leakage kaise prevent hota hai?”

Hint: `features_node` mein sirf row-wise transforms; impute/scale/encode sklearn `Pipeline` ke andar rehte hain, per fold fit hote hain.

3.5 “Held-out metrics honestly kaise compute hote hain?”

Hint: `evaluate_model` re-split karta hai (`TEST_SIZE=0.2`), pipeline ko `clone` karta hai, sirf train pe refit karta hai, held-out split pe score karta hai.

3.6 “`explain_model` ye kaise guarantee karta hai ki run kabhi break nahi hoga?”

Hint: Guarded `import shap` ; `TreeExplainer/LinearExplainer` → permutation fallback → method `"unavailable"` ; kabhi raise nahi karta.

3.7 “`cv_folds` knob repair actually training tool tak kaise pahunchta hai?”

Hint: `reduce_cv_folds` `artifact_key="cv_folds"` set karta hai; `reflect_node` isko `problem_spec` mein deep-copy karta hai; `training_node` `spec.get("cv_folds")` read karta hai.

4. Ye kyun, wo nahi (trade-offs)

4.1 “LLM se code likhwane ke bajaye deterministic tools se ML kyun karwaya?”

Hint: Reproducible, testable, koi arbitrary code execution nahi; LLM decisions validate karna cheap hai.

4.2 “Hamesha LLM se poochne ke bajaye heuristic-first diagnosis kyun?”

Hint: Common failures cheap/deterministic/offline hote hain; LLM sirf uncertain tail ke liye.

4.3 “Failure pe bas re-run karne ke bajaye SQLite checkpointing kyun?”

Hint: Last good node se resume karo; expensive training dobara mat karo; par ye single-machine hai (trade-off).

4.4 “Intermediates ke liye CSV ya in-memory ki jagah Parquet kyun?”

Hint: Typed + compact + retry pe fast re-reads; repairs isko same path pe in-place rewrite karte hain.

4.5 “Recommendations 6 pe aur retries chhote numbers (3/2/2/4) pe kyun cap kiye?”

Hint: Bounded budgets termination guarantee karte hain; `recursion_limit` inse upar sized hai final backstop ke tor par.

5. Failure aur edge cases (code se verify kiye hue)

5.1 “User ek file upload karta hai jo actually CSV nahi hai — kya hota hai?”

Hint: `datasets.py` accept karne se pehle `profile_dataset` se parse karta hai; parse fail → reject aur delete, koi run create nahi hota.

5.2 “Target column Yes/No strings hai aur training crash ho jaati hai — phir?”

Hint: Catch hota hai → `TARGET_ENCODING_ERROR` → `label_encode_target` 0/1 mein rewrite karta hai → same step retry hota hai → complete hota hai.

5.3 “Ek repair chalta hai par kuch change nahi karta — kya ye hamesha retry karta rehta hai?”

Hint: Nahi; `applied && changed` gate hai — no-op retry burn karne ke bajaye escalate karta hai; `test_noop_repair_escalates_without_retry` test hai.

5.4 “Reflection fix nahi kar paata — kya failure kabhi intelligently re-plan hota hai?”

Hint: Honestly, abhi nahi — `handle_escalation` same plan ko retry karta hai phir `needs_input` pe stop ho jaata hai; hint record hota hai, act nahi hota.

5.5 “Planner LLM baar baar invalid plan emit karta rehta hai — kya request hang ya 500 ho jaati hai?”

Hint: Dono nahi; `plan_attempts=3` ke baad ye validator errors ke saath `needs_input` return karta hai; node exception ek `failed` `run.json` banata hai, HTTP 500 nahi.

6. Scaling aur production

6.1 “Ye poori HTTP request ko block karta hai — kaise fix karoge?”

Hint: Background worker / job queue; `POST /runs` turant ek id return kare; `GET /runs/{id}` poll karo; run record already ye model karta hai.

6.2 “Ise multiple machines pe kaise run karoge?”

Hint: Abhi single-process hai (SQLite checkpoints + local `storage/`); checkpoints ko shared DB mein aur artifacts ko object storage mein move karo.

6.3 “Dataset 5 GB ka hai — kaha fail hoga, aur kya change hoga?”

Hint: Sab kuch pandas mein in-memory load hota hai; chunking/out-of-core ya sampling step chahiye; `max_upload_mb=200` abhi ise cap karta hai.

6.4 “Production mein ise kaise observe karoge?”

Hint: `reflection_history` already ek audit trail hai; repair frequency, escalation rate, per-node latency pe metrics add karo.

6.5 “Escalation branch ko actually useful kaise banaoge?”

Hint: `planner_hint` pe act karo — model swap, search space shrink, plan edit — seam already `_handle_escalation` mein hai.

7. Reflection

7.1 “Sabse zyada proud kis part pe ho, aur kyun?”

Hint: Diagnose→repair→retry loop, clean repair-vs-escalate boundary, aur bounded budgets ke saath.

7.2 “Agar phir se banate toh kya rebuild karte?”

Hint: Shuru se hi async runs; aur escalation ko hint pe act karwate, sirf retry nahi.

7.3 “Kya surprising ya expected se harder tha?”

Hint: Self-fix loop ko khud loop hone se rokna — teen budgets + `recursion_limit` backstop isi ke liye hain.

7.4 “Agar ek aur hafta milta, sabse high-value cheez kya add karte?”

Hint: Background job queue (API unblock karega) ya escalation pe real replan; ek pick karo aur justify karo.